



Attention Mechanism





Transformer

- A deep learning model that adopts the mechanism of self-attention, differentially weighting the significance of each part of the input data.
- Used primarily in the fields of natural language processing (NLP) and computer vision (CV).
- Designed to process sequential input data, such as natural language. Unlike RNNs, transformers process the entire input all at once (parallelism)
- Attention mechanism provides context for any position in the input sequence.
- Leads to pretrained systems such as
 - BERT (Bidirectional Encoder Representations from Transformers) and
 - GPT (Generative Pre-trained Transformer),





Applications of Transformer

- machine translation
- document summarization
- document generation
- named entity recognition (NER):
 - An NLP task extracting information from unstructured text
- biological sequence analysis
- writing computer code based on requirements expressed in natural language.
- video understanding.



[https://en.wikipedia.org/wiki/Transformer_\(machine-learning_model\)](https://en.wikipedia.org/wiki/Transformer_(machine-learning_model))



What is Attention?

- The core idea behind the Transformer model is the attention mechanism
- In early sequence-to-sequence models for machine translation, the entire input sentence was compressed by the encoder into a single fixed-length context vector as the output of the encoder,
- During decoding, the same context vector representing a compressed representation of the input sentence will be a constant input while the decoder is decoding the output
- Attention mechanism allows decoder to revisit the input sequence at each time step of decoding an output token, while dynamically “attend” to different part of the input sentence at different time step.





Sequence to Sequence Machine Translation

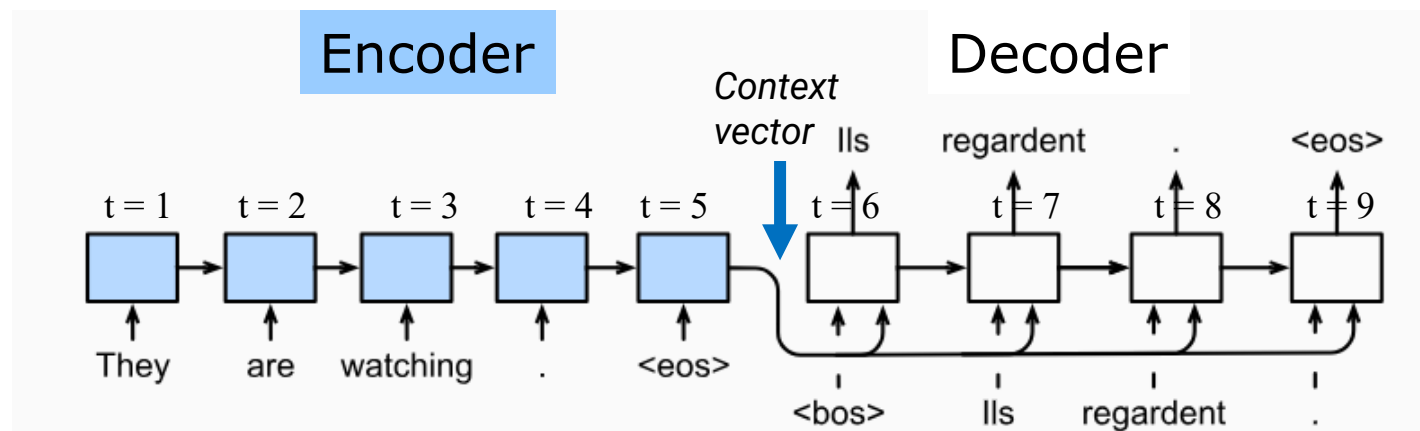


Fig. 10.7.1 Sequence-to-sequence learning with an RNN encoder and an RNN decoder

- <eos>, <bos>: end and beginning of a sequence of token embeddings
- Left: source language, right: target language
- The context vector from encoder at time step 5 will be available to the decoder to ALL time steps
- Attention mechanism facilitate computing a context vector for each decoder state as a weighted combination of ALL encoder states.





Outline of Attention Models

- 11.1. Queries, Keys, and Values
- 11.2. Attention Pooling by Similarity
- 11.3. Attention Scoring Functions
- 11.4. The Bahdanau Attention Mechanism
- 11.5. Multi-Head Attention
- 11.6. Self-Attention and Positional Encoding



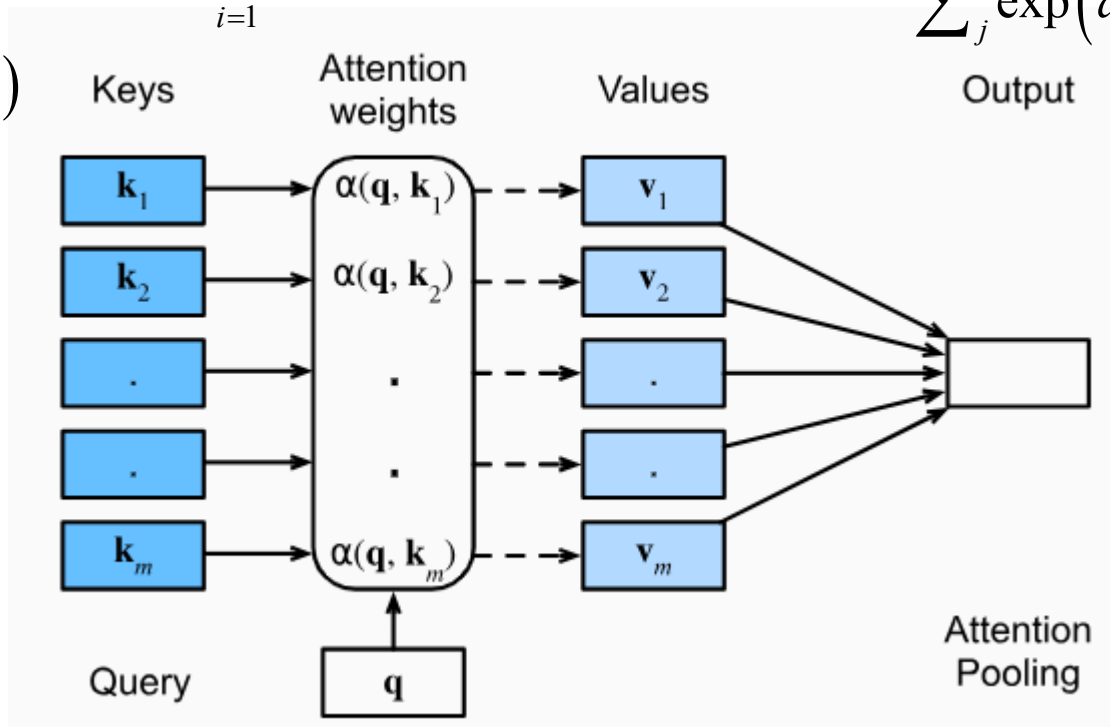


Attention, Query, Key, Value

$$\text{Attention}(\mathbf{q}, D) \triangleq \sum_{i=1}^m \alpha(\mathbf{q}, \mathbf{k}_i) \mathbf{v}_i$$

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_j \exp(a(\mathbf{q}, \mathbf{k}_j))} \quad (11.1.3)$$

(11.1.1)

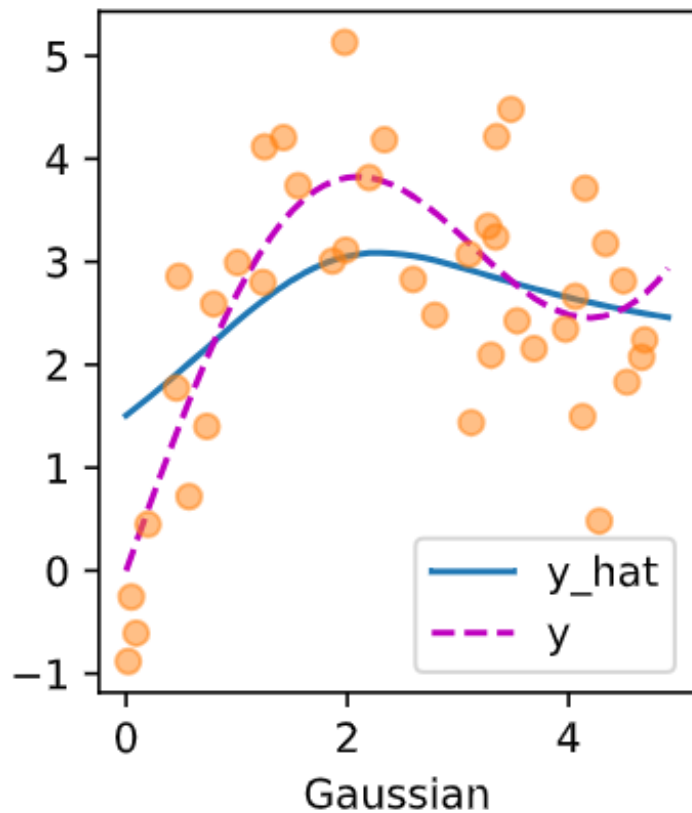


key	value
firstName	Bugs
lastName	Bunny
location	Earth

Fig. 11.1.1 The attention mechanism computes a linear combination over values $\mathbf{v}_i$ via attention pooling, where weights are derived according to the compatibility between a query $\mathbf{q}$ and keys $\mathbf{k}_i$. D : a database of m (key, value) pairs



Attention Pooling



Attention Scoring Function:

$$a(x, x_i) = \exp\left(-0.5w^2(x - x_i)^2\right)$$

Use Nonparametric Nadaraya-Watson regression:

$$y = f(x) \Rightarrow y_i = f(x_i) + \varepsilon_i, 1 \leq i \leq n$$

w : a learnable parameter

x_i : i^{th} keys

x : the query

Attention weight

$$f(x) = \sum_{i=1}^n \alpha(x, x_i) y_i$$

$$= \sum_{i=1}^n \left[\frac{\exp\left(-0.5w^2(x - x_i)^2\right)}{\sum_{j=1}^n \exp\left(-0.5w^2(x - x_j)^2\right)} \right] y_i$$

$$= \sum_{i=1}^n \text{softmax}(a(x, x_i)) y_i$$



Attention Scoring Function

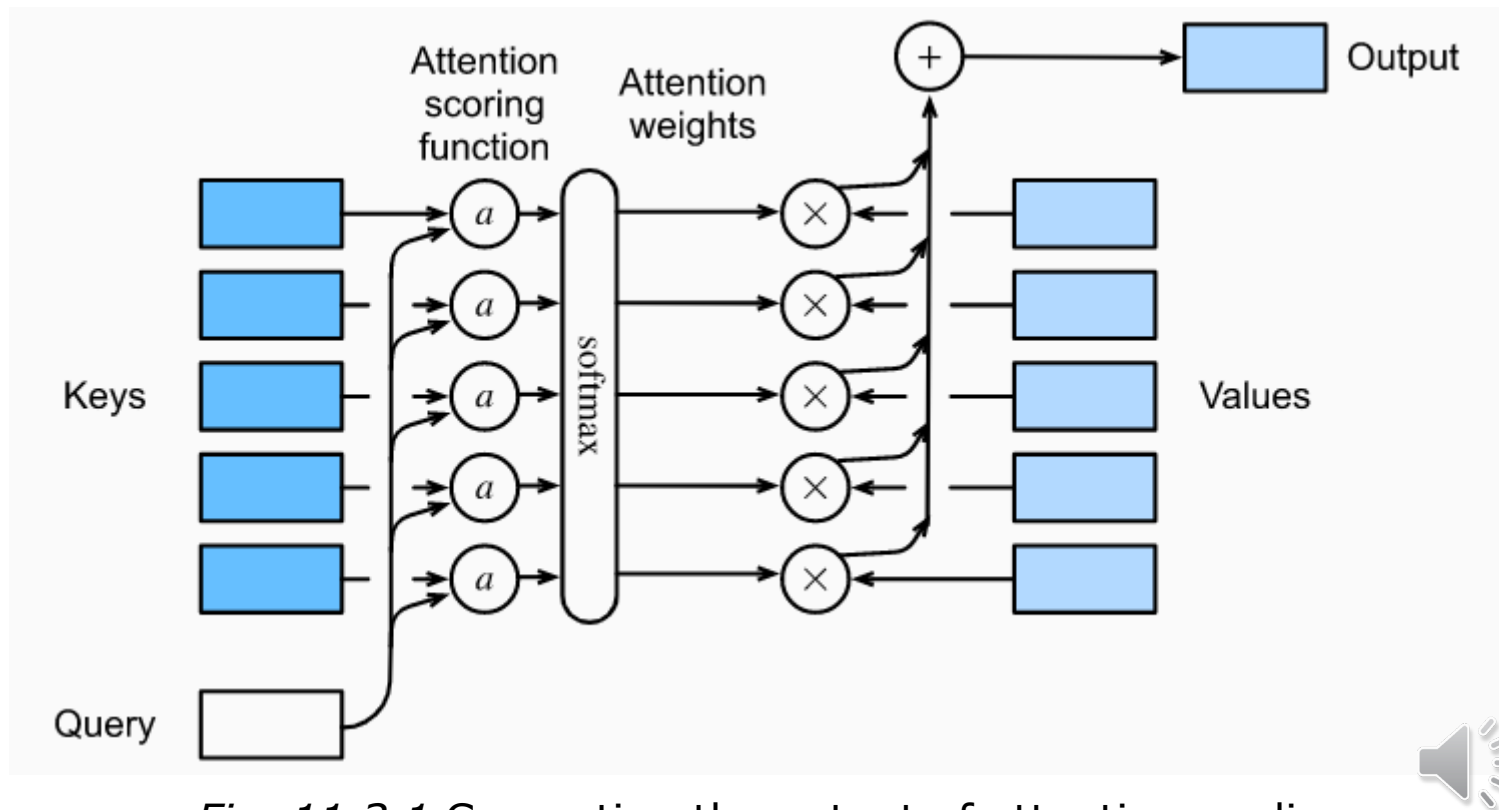


Fig. 11.3.1 Computing the output of attention pooling as a weighted average of values.

Attention Scoring Function

A choice of the attention Scoring function $a(\mathbf{q}, \mathbf{k}_i)$ is the scaled dot product

$$a(\mathbf{q}, \mathbf{k}_i) = \mathbf{q}^T \mathbf{k}_i / \sqrt{d} \quad (11.3.2)$$

where d is the query dimension. One may normalize it using the softmax function to compute the attention weights

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_{j=1}^m \exp(a(\mathbf{q}, \mathbf{k}_j))} = \text{Soft max}(a(\mathbf{q}, \mathbf{k}_i)) \quad (11.3.3)$$





Masked Softmax Scoring Function

Mask: $M \subset \{1, 2, \dots, m\}$ consists of indices that should be used for the softmax.

$$\alpha_{\text{masked}}(\mathbf{q}, \mathbf{k}_i) = \begin{cases} \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_{j \in M} \exp(a(\mathbf{q}, \mathbf{k}_j))} & i \in M \\ 0 & i \notin M \end{cases}$$

Additive Attention Scoring Function (when $\mathbf{q}$ and $\mathbf{k}$ have different dim)

$$a(\mathbf{q}, \mathbf{k}_i) = \mathbf{w}_v^T \tanh(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k}_i) \quad (11.3.7) \img alt="speaker icon" data-bbox="850 800 900 860"/>$$

used when $\mathbf{q}$ and $\mathbf{k}_i$ are of different lengths